

CENA

WhatsApp Bot

Manual de Instalação

Guia completo de configuração do ambiente com Evolution API, PostgreSQL, Redis, FastAPI, Groq e Ngrok.

FLUXO DE FUNCIONAMENTO



Visão Geral

Este documento descreve o processo de instalação e configuração do ambiente necessário para executar o CENA WhatsApp Bot. A solução integra os seguintes componentes:

Componente	Função
Evolution API	Gateway de integração com o WhatsApp
PostgreSQL	Banco de dados relacional para persistência
Redis	Cache e fila de mensagens
FastAPI	Backend da aplicação em Python
Groq	Modelo de linguagem (LLM) para geração de respostas
Ngrok	Tunelamento HTTP para exposição do backend na internet

1 Configuração do Arquivo .env

Crie um arquivo .env na raiz do projeto com as variáveis abaixo, organizadas por grupo:

Aplicação

```
AUTHENTICATION_API_KEY=cenatestkey
EVO_BASE_URL=http://localhost:8080
EVO_INSTANCE_NAME=cenatest
GROQ_API_KEY=<sua-chave-groq>
OPENROUTESERVICE_API_KEY=<sua-chave-openrouteservice>
DATABASE_URL=postgresql+asyncpg://postgres:postgres@postgres:5432/cenatest
DATABASE_URL_SYNC=postgresql://postgres:postgres@postgres:5432/cenatest
```

PostgreSQL

```
POSTGRES_USER=postgres
POSTGRES_PASSWORD=postgres
POSTGRES_DB=cenatest
```

Evolution API — Banco de Dados

```
DATABASE_ENABLED=true
DATABASE_PROVIDER=postgresql
DB_CONNECTION=postgresql
DATABASE_CONNECTION_URI=postgresql://postgres:postgres@postgres:5432/cenatest?schema=public
DATABASE_CONNECTION_CLIENT_NAME=evolution_exchange
DATABASE_SAVE_DATA_INSTANCE=true
DATABASE_SAVE_DATA_NEW_MESSAGE=true
DATABASE_SAVE_MESSAGE_UPDATE=true
DATABASE_SAVE_DATA_CONTACTS=true
DATABASE_SAVE_DATA_CHATS=true
DATABASE_SAVE_DATA_LABELS=true
DATABASE_SAVE_DATA_HISTORIC=true
```

Redis

```
CACHE_REDIS_ENABLED=true
CACHE_REDIS_URI=redis://redis:6379/6
CACHE_REDIS_PREFIX_KEY=cenatestes
CACHE_REDIS_SAVE_INSTANCES=false
CACHE_LOCAL_ENABLED=false
```

pgAdmin

```
PGADMIN_EMAIL=
PGADMIN_PASSWORD=
```

2 Inicializar a Infraestrutura

Suba todos os serviços definidos no Docker Compose com o comando:

```
docker compose up -d
```

Para visualizar os logs em tempo real durante a inicialização:

```
docker compose up
```

Após a inicialização, verifique se os containers estão rodando corretamente:

```
docker ps
```

3 Executar o Backend

Instale as dependências do projeto usando uv:

```
uv sync
```

Inicie a aplicação FastAPI em modo de desenvolvimento:

```
uv run fastapi dev main.py
```

O backend ficará disponível em:

```
http://localhost:8000
```

4 Expor a Aplicação com Ngrok

Abra um novo terminal e execute o Ngrok para criar um túnel público:

```
ngrok http 8000
```

O Ngrok irá gerar uma URL pública semelhante a:

```
https://corban-nonsupposing-bret.ngrok-free.dev
```

⚠ Importante: Guarde esta URL pois ela será utilizada na configuração do Webhook na Seção 9.

5 Configurar a Evolution API

Acesse o painel de gerenciamento da Evolution API pelo navegador:

```
http://localhost:8080/manager
```

Na tela de login, utilize as seguintes credenciais:

Campo	Valor
URL	http://localhost:8080
API Key	cenatestekey

Clique em Login para acessar o painel.

6 Criar uma Nova Instância

Dentro do painel da Evolution API:

1. Clique em New Instance.
2. Informe o nome da instância. Utilize exatamente o valor configurado em EVO_INSTANCE_NAME:

```
EVO_INSTANCE_NAME=cenateste
```

```
Nome da instância: cenateste
```

7 Conectar um Número de WhatsApp

Informe o número de telefone no seguinte formato:

```
55DDD9NUMERO
```

Exemplo:

```
5581992225516
```

Regras de formatação do número:

- Deve iniciar com 55 (código do Brasil)
- Deve conter o DDD com 2 dígitos
- Deve conter o dígito 9 antes do número de 8 dígitos

Após criar a instância, um QR Code será exibido automaticamente na tela.

8 Vincular o WhatsApp

No celular que possui o número informado, siga os passos abaixo:

3. Abra o WhatsApp.
4. Acesse Configurações > Dispositivos Conectados.
5. Clique em Conectar um Dispositivo.
6. Escaneie o QR Code exibido pela Evolution API.

O processo é idêntico ao utilizado pelo WhatsApp Web. Após a leitura do QR Code, a instância estará conectada e pronta para uso.

9 Configurar o Webhook

Dentro da instância criada no painel da Evolution API:

7. Acesse as configurações da instância.
8. Ative a opção Webhook.
9. No campo URL, informe a URL do Ngrok seguida de /webhook:

```
https://corban-nonsupposing-bret.ngrok-free.dev/webhook
```

10. Habilite o evento:

```
MESSAGES_UPSERT
```

11. Clique em Save e aguarde a confirmação.

10 Testar a Integração

Com todas as configurações concluídas, realize o teste de integração:

12. Utilize outro WhatsApp para enviar mensagens ao número cadastrado.
13. Verifique se a mensagem é recebida pela aplicação.

14. Acompanhe os logs do backend (terminal onde está o FastAPI).

15. Acompanhe os logs dos containers Docker:

```
docker compose logs -f
```

Comportamento esperado ao receber uma mensagem:

- A Evolution API recebe a mensagem via WhatsApp
- O webhook é acionado e envia o conteúdo ao backend
- O FastAPI processa e encaminha ao modelo LLM (Groq)
- O Groq gera a resposta automaticamente
- A Evolution API entrega a resposta ao usuário no WhatsApp

Checklist Final

Utilize esta lista para verificar se todos os passos foram concluídos antes de colocar o bot em produção:

☐ Arquivo .env configurado com todas as variáveis
☐ Containers iniciados com docker compose up -d
☐ Backend executando na porta 8000
☐ Ngrok ativo e URL pública gerada
☐ Evolution API acessível em http://localhost:8080/manager
☐ Instância cenatestes criada
☐ WhatsApp conectado via leitura do QR Code
☐ Webhook configurado com a URL do Ngrok
☐ Evento MESSAGES_UPSERT habilitado
☐ Teste de envio de mensagens realizado
☐ Respostas sendo enviadas automaticamente pelo bot